LEAP · PIKPAK

PikPak Fleet Database Plan

One company database in Azure to replace the spreadsheets: the fleet, customers, SKUs, trays, maintenance, issues and root causes, daily production, with every change tracked and backups that look after themselves.

Draft for discussion · 7 September 2026
Prepared by Chris Hamblin.

Summary

Today the fleet's information lives in spreadsheets, Teams chats, or people's heads: system serial numbers, customers, SKU numbers, tray numbers, maintenance events and root cause analyses, each in its own workbook. They are hard to join, hard to back up and impossible to audit.

This plan replaces them with one database in Azure. It is a PostgreSQL database, the same engine behind most modern web systems, run for us as a managed service so backups, patching and failover are Microsoft's job. A small internal web app is where people add and edit records, with Microsoft sign-in and a change history on every row. Excel, Power BI and Grafana read from it. The Logfather reads its master data from it and writes back what each system packed and which software it runs, without anyone typing.

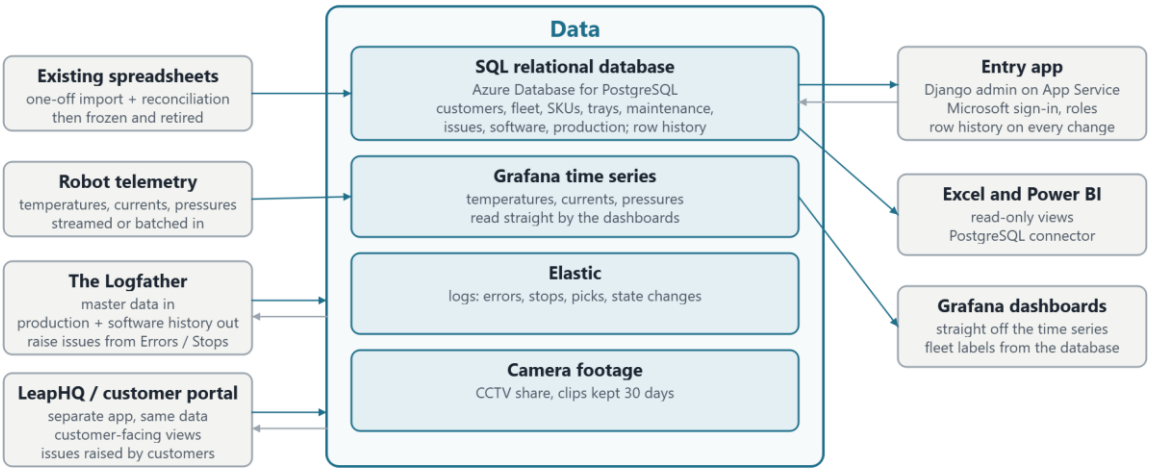
The telemetry that Grafana shows today, component temperatures and motor currents, goes into the same database through the TimescaleDB extension, so a temperature excursion, the servo that was fitted at the time, and the issue it became are three tables apart, not three systems apart.

> **What we are asking for.** Agreement on the eight areas and the tables in them, on the engine, and on the order of work. Azure costs are roughly £100 a month.
>
> **What happens first.** A discovery pass to inventory the spreadsheets and name the data owners, then a short hands-on trial of the managed database before any real infrastructure is built.

How the system works

Three layers, each replaceable on its own.

- **One database, the source of truth.** Azure Database for PostgreSQL holds every record: the relational tables, JSON where a record is naturally a document, and the time series through TimescaleDB. Spreadsheets become read-only views of it, never the other way round.
- **One entry app.** A small internal web app for adding and editing records, with sign-in through the company Microsoft account, roles, and a full change history on every row. Phase one is a generated admin interface, not bespoke screens.
- **Consumers.** Excel and Power BI read from views with a read-only login. Grafana reads the telemetry tables directly. The Logfather reads customers, lines, systems and SKUs from the database, writes production and software history into it, and can raise an issue from its Errors / Stops window with the system and day filled in. LeapHQ, the customer portal being built separately, uses the same database for its customer-facing views and keeps reading Elastic and the CCTV share directly, as the Logfather does.



Solid arrows: data in or out. Grey: reads back. The Logfather and LeapHQ read all four stores: records from the database, readings from the Grafana time series, logs from Elastic, footage from the camera share. One Azure resource group, one Microsoft sign-in, one bill.

Trackable

Every table carries who created and last changed a row and when, and the entry app writes a history row for every change on the core tables. "What did this system's line assignment look like on 3 March" is a single query. Nothing is ever hard-deleted; rows are retired with an end date.

Easy to back up

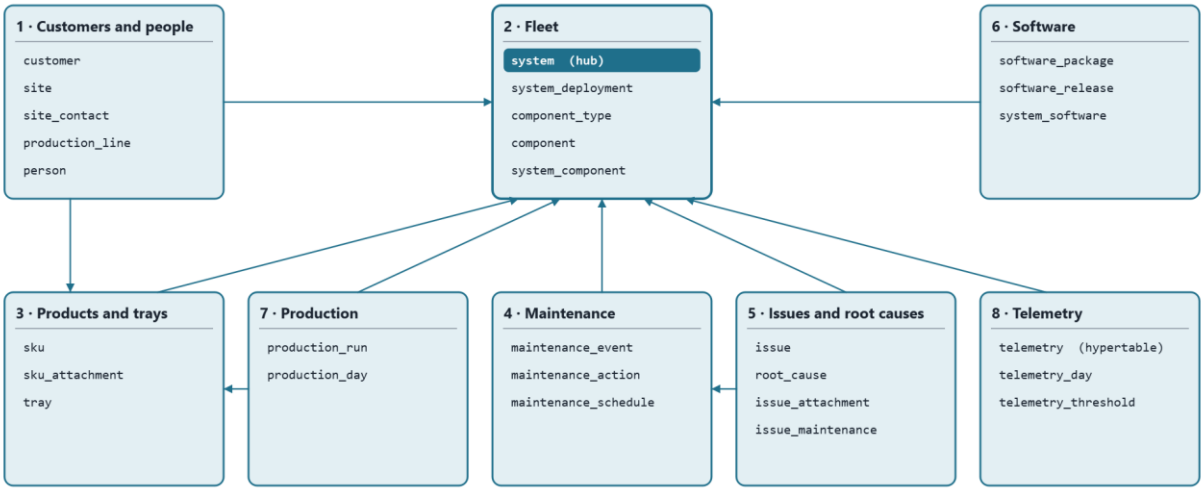
The managed service takes automatic backups with point-in-time restore for up to 35 days, optionally copied to a second region. For years-long retention a weekly dump goes to Blob Storage with a lifecycle rule, one small scheduled job. A restore drill is a ticket in the plan, not an afterthought.

The engine

> **Near term: Azure Database for PostgreSQL for the records; Grafana keeps the time series it holds today.**
>
> **Suggested long-term: Azure Database for PostgreSQL Flexible Server, with the TimescaleDB extension, one database for everything.**
>
> Relational tables for the fleet, JSON columns where a record is naturally a document, and TimescaleDB hypertables for the telemetry, all under one backup, one sign-in and one connection string. Django speaks PostgreSQL natively, Grafana has a first-class PostgreSQL data source, and Excel and Power BI connect through the PostgreSQL connector.
>
> What we give up against Azure SQL: built-in temporal tables (row history comes from the app instead), the Excel SQL Server connector people have seen before (the PostgreSQL one needs a one-time driver install), and built-in long-term backup retention (a weekly dump covers it). What we gain: no second store for telemetry, no cross-store joins, and a smaller bill.

Not recommended: Dataverse or SharePoint lists (licensing per user, weak relational model, hard to query from outside), a separate NoSQL store (this is relational data with many-to-one links everywhere), or a separate time-series service now that Timescale lives inside the same database.

The eight areas



Area	Tables	What it answers
1 · Customers and people	`customer`, `site`, `site_contact`, `production_line`, `person`	Who our customers are, where their sites and lines are, who to call.
2 · Fleet	`system`, `system_deployment`, `component_type`, `component`, `system_component`	Every PikPak, where it is and was, what is fitted in it.
3 · Products and trays	`sku`, `sku_attachment`, `tray`	Every product line, its pictures, and the tray it is packed in. A SKU is the same wherever it is packed, so it belongs to no one customer.
4 · Maintenance	`maintenance_event`, `maintenance_action`, `maintenance_schedule`	What was done to each system, by whom, and what is due.
5 · Issues and root causes	`issue`, `root_cause`, `issue_attachment`, `issue_maintenance`	What went wrong, why, and what fixed it.
6 · Software	`software_package`, `software_release`, `system_software`	Which release each system runs and which are approved.
7 · Production	`production_run`, `production_day`	Which SKU went into which tray on which system on which day, and how many. Fed from Elastic, not typed.
8 · Telemetry	`telemetry`, `telemetry_day`, `telemetry_threshold`	Every temperature, current and pressure reading, and the daily summary per component that issues and maintenance planning use.

Keys and conventions

- **Surrogate primary keys everywhere:** an integer id per table. Serial numbers, tray numbers and SKU codes are human identifiers that get retyped and corrected, so they must not be what other tables point at.

- **Natural keys as unique constraints:** a system's serial number (with the display name "PikPak 007" unique too, but only ever a label), a tray's unique number, a SKU's code, a package's git commit. Duplicates are rejected by the database.
- **Human references** on things people talk about: ISS-0001, MNT-0001, RC-0001, generated from the id and shown everywhere.
- **History as date ranges, not overwrites.** Where a system is, which component is fitted, which software runs: each is a row with from and to dates, and only one open row is allowed per subject.
- **Lookups are tables, not free text:** statuses, severities, event types, component types and issue categories, so a new value is a row, not a code change.
- **Audit on every table** plus a history table per core entity, giving the same "as of" queries a temporal table would.
- **Time series are hypertables:** partitioned by time automatically, compressed after 30 days, kept for years.

The tables

Key: the primary key, then the unique or natural key in brackets. Columns lists the ones worth arguing about; audit columns are on every table.

1 · Customers and people

Table	Key	Columns worth naming
`customer`	`customer_id (name)`	short_code (EVG, LWS), status, account owner, notes
`site`	`site_id (customer_id, name)`	address, timezone, notes
`site_contact`	`site_contact_id (site_id, person_id, role)`	The customer's people at a site: person, role (production manager, maintenance lead, IT, quality, commercial), is_primary, phone / email overrides, active_from, active_to. A person can hold roles at several sites.
`production_line`	`line_id (site_id, name)`	name ("Line 6"), description, shift pattern
`person`	`person_id (email)`	name, phone, organisation (own / customer / contractor), customer_id when they belong to a customer, job title, active. Used by contacts, maintenance and issues alike.

2 · Fleet

Table	Key	Columns worth naming
`system`	`system_id (serial_number; display_name; folder_name)`	The serial number "35-2300-007" is the identity; display_name "PikPak 007" is what every screen shows; folder_name "PikPak007" is the CCTV share folder, kept separate so a rename never breaks the footage path. Plus generation (Argus 1 / 2), build_date, status (in build, FAT, installed, workshop, decommissioned), notes.
`system_deployment`	`deployment_id (one open row per system)`	system, line, installed_on, removed_on, reason. Answers "where was PikPak 007 in March".
`component_type`	`component_type_id (name)`	servo, drive, camera, PSU, IO card, PC, gripper, conveyor motor
`component`	`component_id (serial_number; part_number)`	type, supplier, purchased_on, warranty_until, status
`system_component`	`system_component_id (one open row per system + position)`	system, component, position ("servo 2", "pick camera"), fitted_on, removed_on, the maintenance event that fitted it. Telemetry joins to this row, so a reading is attributed to the exact servo in that slot at the time.

3 · Products and trays

No tray-type table: most trays are unique with subtle differences, so the design lives on the tray. No recipe-version table: a SKU never changes once defined and is the same for every customer that packs it.

Table	Key	Columns worth naming
tray	tray_id (unique_number)	dimensions, cavity layout, material, supplier, drawing reference; customer, site, status (in service, damaged, retired), commissioned_on, retired_on. A damaged tray is an issue with the tray linked, not a separate log.
sku	sku_id (code)	code as logged in Elastic ("EVG_picc…"), description, product dimensions and weight, products per pick, the tray it is packed in, config (JSON, the recipe), status. Not owned by a customer.
sku_attachment	attachment_id	sku, kind (photo, drawing, spec sheet), file in Blob Storage, caption, sort order, uploaded by. One SKU can carry several pictures.

4 · Maintenance

Table	Key	Columns worth naming
maintenance_event	event_id (MNT-0001)	system, type (planned service, breakdown, part replacement, software update, calibration, inspection, FAT), started_at, ended_at, downtime_minutes, performed_by, summary, external ticket
maintenance_action	action_id	event, description, component removed, component fitted, labour hours, parts cost
maintenance_schedule	schedule_id (system_id, task)	task, interval (days or picks), last done, next due. Optional in phase one.

5 · Issues and root causes

Kept deliberately simple: a root cause is an entry in a catalogue with its own number, and an issue points at one. No workflow states, no analysis records; those can be added later.

Table	Key	Columns worth naming
root_cause	root_cause_id (RC-0001; name)	The catalogue of every known root cause: name ("Vacuum solenoid stuck"), description, category (mechanical, electrical, software, operator, product, tray, environment), active. Never deleted, only retired.
issue	issue_id (ISS-0001)	system (nullable for site-level), site, tray (nullable), reported_at, reported_by, title, description, severity, root cause (nullable until known), Grafana link, Jira key, notes
issue_attachment	attachment_id	issue, kind (photo, clip, log export, document), file, caption, uploaded by
issue_maintenance	(issue_id, event_id)	links an issue to the maintenance that fixed it

6 · Software

Table	Key	Columns worth naming
`software_package`	`package_id (name)`	argus, planner, targeting, actuators, sensors, infeed, crate_change, behaviour, firmware packages
`software_release`	`release_id (package_id, git_commit)`	version, branch, released_on, approved, release notes. Where "which commits are approved for the fleet" lives.
`system_software`	`system_software_id (one open row per system + package)`	system, package, release, installed_at, removed_at, source (manual / Elastic). The Logfather already computes this from Elastic.

7 · Production

Table	Key	Columns worth naming
`production_run`	`run_id (elastic_run_id)`	system, SKU, tray, started_at, ended_at, products_packed, trays_packed, products_seen, products_rejected, pick_movements, raw counters (JSON), source. One row per run.
`production_day`	`production_day_id (system, day, sku, tray)`	The daily roll-up: products_packed, trays_packed, products_rejected, pick_movements, run_count, minutes_running, minutes_on. The table reports and Excel use.

8 · Telemetry

Raw readings and their daily summary live in the same database as everything else. The raw table is a TimescaleDB hypertable: an ordinary table partitioned by time behind the scenes, compressed after 30 days, with old chunks dropped past the retention we set.

Table	Key	Columns worth naming
`telemetry`	`(time, system_id, position, metric)`	time, system, position ("servo 2", "brake resistor", "PSU 48V"), metric (temperature, current, voltage, pressure, vacuum), value, unit, source. Compression after 30 days; retention 5 years; indexed so one servo's panel is instant.
`telemetry_day`	`(system_id, position, metric, day)`	the fitted component that day, min, average, max, minutes above threshold, threshold used, sample count. A continuous aggregate refreshed nightly; the table issues and maintenance planning read.
`telemetry_threshold`	`(component_type_id, metric)`	warning and alarm levels per component type and metric, so "minutes above threshold" changes with a row edit, not a code change.

Decisions needed at the meeting

- **The eight areas and their tables.** Anything missing, anything that should not be there. Recent simplifications to confirm: no tray-type table, no SKU recipe versions, SKUs not owned by customers, root causes as a numbered catalogue, damaged trays as issues.
- **The engine.** The suggestion is PostgreSQL with TimescaleDB, one database for records and telemetry, in place of Azure SQL plus a separate time-series service.
- **Identifiers.** Serial-number format, tray numbering, that SKU codes come from Elastic's SKU name, and the ISS / MNT / RC reference scheme.

Decisions needed soon

- **Data owners.** A name against each area for the inventory, the reconciliation and the sign-off.
- **Roles.** Who may create systems, close issues, approve releases; who is read-only.
- **Where Grafana reads from today,** its retention and how far back data still exists, since that decides how telemetry gets in.